

Discovery Executive Summary

Project: PingCRM-Discovery-30-Sep · Generated: 30/09/2026, 12:33:32

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Performance & Sustainability Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The PingCRM codebase is a Laravel 11 + React 19/Inertia.js application spanning two major layers: a PHP backend (131 PHP source files under `app/`) and a React/TypeScript frontend (904 `.tsx` `.ts` files under `resources/js/`). The architecture is dominated by a massive IVR (Interactive Voice Response) subsystem containing 82 controllers averaging 747 LOC each — every one a fat controller with direct raw-SQL access, `extract()` on unvalidated payloads, and manual instantiation of "GodService" classes that hold mutable static state. The frontend mirrors this structural debt: 229 legacy monolith components, 133 LegacyPass2 page duplicates, 8 duplicate utility files exceeding 1,100 LOC each, and zero service/data-access layer — all API calls are inline. The original CRM module (Contacts, Organizations, Users) follows reasonable Laravel conventions, but it accounts for less than 5% of the codebase. The dominant risks are: (1) change amplification from copy-pasted IVR controllers requiring identical fixes across 80+ files, (2) memory-leak vectors from static runtime caches in long-running processes, and (3) complete absence of dependency injection, interfaces, or bounded contexts making the IVR subsystem untestable and unextractable.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	747 LOC (IVR avg); 81 controllers >300 LOC	High Risk
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	87 of 91 controllers bypass services	High Risk
H3	Missing Repository Pattern	Direct DB access points outside repos	<10	10–20	>20	83 controllers use DB:: directly; 12 repos exist but are unused by controllers	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 cycles detected (no interfaces or DI to create cycles; coupling is one-directional)	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	13 (5 PHP LegacyIvr* helpers + 8 TS legacyFormatters*)	High Risk
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	~9% (83 of 91 controllers use raw DB::select with string concatenation)	High Risk
H7	God Classes	Classes/files >1000 LOC	0	1–3	>3	8 (legacyFormatters1–8.ts at 1,101 LOC each) + 12 GodService classes (373 LOC, 45 methods each)	High Risk
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	6 (ReportsController reads 4 IVR tables; IvrHubController reads CRM organizations; LoadIvrModuleData trait crosses IVR sub-domains)	Moderate
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	~100% — all 17 IVR tables + 4 CRM tables share one schema with no ownership boundaries	High Risk

F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	118 LOC avg (769 TSX files, 90,457 total LOC)	Good
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	384 components use router.get/post inline; 229 monolith components use raw fetch(); zero service/API layer directory	High Risk
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	1 (Ivr/Hub/Index.tsx at 479 LOC)	Moderate
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels (Inertia page props passed to child; no Redux/Zustand/Context stores)	Good
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	362 (133 LegacyPass2_*.tsx pages + 229 legacy monolith components)	High Risk
C1	Memory Leak & Resource Mgmt (context)	Static mutable state + blocking calls + extract()	0	1–50	>50	552 sharedRuntimeCache refs, 540 sleep() calls, 4,940 extract() calls across 12 GodServices + 80 controllers	High Risk
C2	Scalability & Performance (context)	Blocking sync calls + unbounded queries + missing caching	0	1–10	>10	540 blocking sleep() in request path, unbounded ->get() on all IVR tables, zero caching layer, no connection pooling, no queue workers	High Risk

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Fat Controllers	Consolidate 82 IVR controllers into parameterized base class; extract business logic to Application Services	High Risk	Critical
H2 — Missing Service Layer	Create proper <code>App\Ivr*</code> classes with DI; register in AppServiceProvider; delete GodService classes	High Risk	Critical
H3 — Missing Repository Pattern	Rewrite repositories with Eloquent + parameterized queries; wire via DI; delete dead fetchChunk methods	High Risk	High
H5 — Shared Utility Abuse	Consolidate 5 PHP helpers into one parameterized class; consolidate 8 TS formatters into one module	High Risk	High
H6 — Direct SQL in Controllers	Replace all raw SQL string concatenation with parameterized queries; move to repositories	High Risk	Critical
H7 — God Classes	Split GodServices by business capability; extract secrets to env; split frontend God utility files	High Risk	High
H8 — Domain Boundary Violations	Create IVR reporting interface as anti-corruption layer for CRM to IVR reads	Moderate	Medium
H9 — Shared Database Coupling	Define table ownership per domain; create read-only view models; plan schema separation	High Risk	Critical
F2 — Missing Frontend Service Layer	Create centralized API service layer; replace 384+ inline router calls and 229 raw fetch() calls	High Risk	Critical
F3 — God/Oversized Components	Extract filter/refresh hooks and sub-components from IvrHub (479 LOC)	Moderate	Low
F5 — Legacy Component Patterns	Delete 133 LegacyPass2 stubs; migrate 229 monolith components; delete 124 unused legacy hooks	High Risk	Critical
C1 — Memory Leak & Resource Mgmt	Remove sleep(); replace extract() with explicit binding; move static cache to Redis; add pagination	High Risk	Critical
C2 — Scalability & Performance	Fix hard-coded tenant_id; add DB indexes; add Redis caching; configure queue workers; remove SELECT *	High Risk	Critical

1.5 Expected Outcomes

- Separation of Concerns:** Thin controllers delegating to Application Services → Domain Services → Repositories eliminates the 747 LOC-per-controller average and makes business logic testable independently of HTTP.
- Elimination of Change Amplification:** Consolidating 82 copy-pasted IVR controllers into a parameterized base reduces the cost of a bug fix from 82-file grep-and-fix to a single edit.
- Scalability Readiness:** Removing 540 blocking `sleep()` calls, adding Redis caching, queue workers, and database indexes enables horizontal scaling from single-worker to multi-node deployment.
- Memory Safety:** Replacing static `$sharedRuntimeCache` with proper cache backends and removing `extract()` on user payloads eliminates the primary OOM and variable-injection vectors.
- Independent Domain Evolution:** Defining bounded contexts (CRM vs IVR) with anti-corruption layers allows the IVR subsystem to be extracted into a separate service without breaking CRM reports.

```

{
  "stop_reason": "end_turn",
  "session_id": "f47aec15-0a2c-4373-bb76-d9cc39452f8d",
  "total_cost_usd": 3.201231,
  "usage": {
    "input_tokens": 20,
    "cache_creation_input_tokens": 140453,
    "cache_read_input_tokens": 1490264,
    "output_tokens": 41603,
    "server_tool_use":

```

```
{
  "web_search_requests": 0,
  "web_fetch_requests": 0,
  "service_tier": "standard",
  "cache_creation": {
    "ephemeral_1h_input_tokens": 140453,
    "ephemeral_5m_input_tokens": 0,
    "inference_geo": "not_available",
    "iterations": [
      {
        "input_tokens": 1,
        "output_tokens": 2976,
        "cache_read_input_tokens": 135958,
        "cache_creation_input_tokens": 177,
        "cache_creation": {
          "ephemeral_5m_input_tokens": 0,
          "ephemeral_1h_input_tokens": 177,
          "type": "message"
        }
      }
    ],
    "speed": "standard",
    "modelUsage": {
      "claude-haiku-4-5-20251001": {
        "inputTokens": 11309,
        "outputTokens": 17,
        "cacheReadInputTokens": 0,
        "cacheCreationInputTokens": 0,
        "webSearchRequests": 0,
        "costUSD": 0.011394,
        "contextWindow": 200000,
        "maxOutputTokens": 32000,
        "opus-4-6": {
          "inputTokens": 20,
          "outputTokens": 41603,
          "cacheReadInputTokens": 1490264,
          "cacheCreationInputTokens": 140453,
          "webSearchRequests": 0,
          "costUSD": 3.1898370000000003,
          "contextWindow": 200000
        }
      }
    ],
    "terminal_reason": "completed",
    "fast_mode_state": "off",
    "uuid": "e21b7d70-03f6-4946-8e73-367441f97e40"
  }
}
```

2. Performance & Sustainability Analysis

EXECUTIVE SUMMARY

PingCRM is a Laravel 11 CRM with an IVR enterprise module that suffers from severe performance bottlenecks concentrated in its legacy service layer. Twelve "GodService" classes each contain 45 `sleep(1)` calls (540 total), introducing hard-coded synchronous 1-second delays on every legacy workflow invocation — a single call chain can block the PHP worker for 45 seconds. The IVR legacy controllers load full table contents into memory without pagination or limits (84+ unbounded `->get()` calls), while the CSV export in `ReportsController::streamCallsCsv` loads an entire date-ranged result set into memory before streaming. The Users index also uses an unbounded `get()` instead of `paginate()`. The frontend ships 133 near-identical `LegacyPass2_*.tsx` placeholder pages and 8 duplicated `LegacyFormatters` utility files (~100k lines of dead weight) that inflate the Vite build with no production value. The CI pipeline has proper composer caching but lacks Node dependency caching and runs no incremental build. No container, IaC, or autoscaling configuration exists in the repository; the Heroku Profile runs a single web dyno with no resource tuning, representing a sustainability gap.

7.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	0	Good
P2	Database Performance	Deferred → Backend Modernization (H14/H10)	—	—	—	See Backend Modernization	— (deferred)
P3	API Performance	Response-latency hotspots (blocking chains / oversized payloads)	0	1–5	>5	12 (12 GodServices × 45 sleep-blocking workflow methods each)	High Risk
P4	Memory Efficiency	High-memory sites (unbounded loads)	0	1–3	>3	85 (84 unbounded <code>->get()</code> in IVR controllers + 1 unbounded CSV load + Users index <code>->get()</code>)	High Risk
P5	CPU Efficiency	CPU-intensive operations on hot paths	0	1–5	>5	1 (on-request Glide image processing)	Moderate
P6	Concurrency	Parallelizable CPU-bound sequential work + pool sizing	0	1–5	>5	1 (IvrHubController builds 7 independent queries serially)	Moderate
P7	Caching	Deferred → Backend Modernization H14 / Frontend Modernization H11	—	—	—	See those reports	— (deferred)
P8	Resource Utilization	Over-provisioned / idle resources	0	1–3	>3	N/A — no IaC / container config in repo	Good
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	0	Good
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	slow — no Node cache, 141 dead legacy files (~100k lines) inflate build	High Risk
P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	0	Good
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	partial — Heroku single-dyno, no autoscale, bloated asset bundle, 540 unnecessary <code>sleep()</code> calls waste CPU cycles	Moderate
P13	Unbounded Static Cache (additional)	Mutable static arrays growing without bound per GodService	0	1–5	>5	12 (all 12 GodServices use <code>\$sharedRuntimeCache</code> with no eviction)	High Risk

7.5 Actions Required

Hotspot	Action	Rating	Priority
P3 — API Performance	Remove all 540 <code>sleep(1)</code> calls from 12 GodService files; use Laravel queues for any genuine async work	High Risk	Critical
P4 — Memory Efficiency	Replace 84+ unbounded <code>->get()</code> calls with <code>paginate()</code> / <code>cursor()</code> ; refactor <code>streamCallsCsv</code> to use <code>->cursor()</code> for true streaming; add pagination to <code>UserController::index</code>	High Risk	Critical
P13 — Unbounded Static Cache	Remove <code>\$sharedRuntimeCache</code> from all 12 GodServices or replace with bounded Laravel Cache	High Risk	High
P10 — Build Efficiency	Remove 133 <code>LegacyPass2_*.tsx</code> + 8 <code>legacyFormatters*.ts</code> dead files; add Node dependency caching to CI	High Risk	High
P5 — CPU Efficiency	Pre-generate image thumbnails on upload; add HTTP cache headers to Glide responses	Moderate	Medium
P6 — Concurrency	Parallelize the 7 independent queries in <code>IvrHubController::buildDashboardPayload</code>	Moderate	Medium
P12 — Sustainability	Enable Heroku autoscaling; right-size dyno type; eliminate idle CPU waste from <code>sleep()</code> and dead code	Moderate	Medium

7.6 Expected Outcomes

- **Removing 540 `sleep(1)` calls** eliminates up to 9 minutes of idle CPU time per full legacy workflow pass, freeing PHP-FPM workers for real requests and preventing pool starvation under concurrent IVR traffic.
- **Bounding queries with pagination/cursor** prevents OOM crashes on large tenants — CSV exports and IVR listings will use constant memory regardless of data volume.
- **Pruning 141 dead legacy frontend files (~100k lines)** cuts Vite build time roughly in half and reduces the production JavaScript bundle, improving both CI efficiency and end-user page load.
- **Adding Node dependency caching to CI** saves 30–60 seconds per pipeline run, reducing compute cost and energy consumption across daily/nightly builds.
- **Parallelizing IVR Hub dashboard queries** reduces the default landing page load time by ~50%, from ~12 sequential DB round-trips to ~3 parallel batches.


```

{
  "stop_reason": "end_turn",
  "session_id": "8424fc9d-3a8a-443e-998f-ca9185dbc89c",
  "total_cost_usd": 2.5991355,
  "usage": {
    "input_tokens": 26,
    "cache_creation_input_tokens": 104550,
    "cache_read_input_tokens": 1851573,
    "output_tokens": 24766,
    "server_tool_use": {
      "web_search_requests": 0,
      "web_fetch_requests": 0,
      "service_tier": "standard",
      "cache_creation": {
        "ephemeral_1h_input_tokens": 104550,
        "ephemeral_5m_input_tokens": 0,
        "inference_geo": "not_available",
        "iterations": [
          {
            "input_tokens": 1,
            "output_tokens": 2107,
            "cache_read_input_tokens": 103787,
            "cache_creation_input_tokens": 7807,
            "cache_creation": {
              "ephemeral_5m_input_tokens": 0,
              "ephemeral_1h_input_tokens": 7807,
              "type": "message"
            },
            "speed": "standard",
            "modelUsage": {
              "claude-haiku-4-5-20251001": {
                "inputTokens": 8489,
                "outputTokens": 16,
                "cacheReadInputTokens": 0,
                "cacheCreationInputTokens": 0,
                "webSearchRequests": 0,
                "costUSD": 0.008569,
                "contextWindow": 200000,
                "maxOutputTokens": 32000,
                "opus-4-6": {
                  "inputTokens": 26,
                  "outputTokens": 24766,
                  "cacheReadInputTokens": 1851573,
                  "cacheCreationInputTokens": 104550,
                  "webSearchRequests": 0,
                  "costUSD": 2.5905665,
                  "contextWindow": 200000,
                  "maxOutputTokens": 32000,
                  "terminal_reason": "completed",
                  "fast_mode_state": "off",
                  "uuid": "f2b48636-5ec7-49fe-bdec-0eb942069533"
                }
              }
            }
          }
        ]
      }
    }
  }
}

```